万卡GPU集群运维全掌握：精通监控四平面、故障五分类与调优三板斧

原创 PingM 基础架构师之路 2026年10月25日 09:00 陕西

公众号 · **基础架构师之路** | 2026-10-25 | AI 硬件架构 · 第 45 篇

导读

这个系列写到第十篇，前面九篇把 RoCEv2 网络怎么选、怎么配、各家方案差在哪讲了一遍：华为的 iLossless、华三的 DDC、NVIDIA 的 Spectrum-X，到第八篇 Juniper、第九篇八家横评。但配好无损网络只是让集群"能跑起来"，**真正吃掉算力的在后面**——几万张卡在几个月里不停地出故障、掉速、Hang 住，而你要在没有用户配合、看不到业务代码的情况下，判断这一次的锅是硬件、是网络还是调度。

这篇按五个问题展开：先立监控（四个平面分别看什么、采样间隔该怎么定），再讲故障五分类和每一类的定位命令，然后把 RoCE 网络的三个典型病单独拆开，之后给调优的三板斧，最后落一份日常巡检清单，顺带把这个系列做个总结。

开场先给一个真实场景。某集群上一个 2048 卡预训练任务，MFU 长期在 38% 上下波动，团队查了两周网络：PFC 计数正常、ECN 标记率正常、交换机上零丢包、ib_write_bw 端到端带宽也达标。最后在 DCGM 的 SM Clock 指标上发现问题——256 台机器里有一台的 8 张卡，SM 时钟长期跑在 1.1GHz，其他机器都是 1.9GHz。这台机器是慢节点，它在每一个 AllReduce 里拖后腿，整条 ring 都要等它。换掉之后 MFU 回到 45%，而且不再波动。**这类问题在交换机上永远查不到**，因为网络侧所有计数都是干净的。

边界说明：文中命令以 NVIDIA DCGM 与 nvidia-smi、Mellanox/NVIDIA OFED、Prometheus 体系为例，交换机侧命令沿用前几篇的厂商命令体系；字段编号、默认值、阈值随驱动、固件、NCCL 与交换机版本变化，请以对应版本的官方文档与现网实测为准，先在测试环境验证。

一、先立监控：四个平面

监控做不好的根因通常不是"指标不够多"，而是"层次没分开"。**GPU 集群的指标天然分四个平面：任务、GPU、网络、链路。四个平面各管一件事，出问题时从上往下逐层排除，能省掉大量反复。**

先看一张分工表，后面每一层都按这张表展开。



GPU 集群监控四平面：任务平面看吞吐与掉队，GPU 平面看利用率与 Xid，网络平面看 PFC/ECN/队列/headroom，链路平面看 CRC/光功率/翻转/拓扑。图要看的是排查顺序——自上而下逐层排除，不要一上来就调门限。 图源：自制

表 1　监控四平面的分工与采集源。每层只回答一个问题，层次混在一起的看板没有排查价值。

平面	回答什么问题	核心指标	主要采集源
任务	任务跑得快不快、有没有卡住	MFU、samples/sec、单 step 耗时、loss 曲线、日志最后更新时间	用户日志锚点、K8s Event、Prometheus
GPU	卡有没有在用、有没有坏	Utilization、SM Active、SM Occupancy、Tensor Active、SM Clock、显存占用、Xid、NVLink 带宽	DCGM / dcgm-exporter、nvidia-smi、dmesg
网络	无损机制有没有生效	PFC 暂停帧计数、ECN 标记率、队列深度、headroom 余量、CNP Slow Restart、RDMA 重传	交换机计数、ethtool -S、rdma stat
链路	物理层是不是干净的	CRC 错包、收发光功率、BER、端口翻转次数、LLDP 邻居、卡间拓扑	ethtool -S/-m、display interface、nvidia-smi topo -m

任务平面：不看业务代码也能判断快慢

任务平面是大盘的第一行。**它要给三个数：MFU、单 step 耗时、日志最后更新时间。** MFU 是唯一能把硬件、框架、并行策略的差异都归一化掉的可比指标，跨集群、跨模型都能横向比；单 step 耗时的方差比均值更有意义——方差大说明有 Straggler；日志最后更新时间是最便宜的 Hang 检测手段，任务日志超过 5 分钟没有新输出就告警，比任何探针都早发现 NCCL Hang。

任务平面的第二个抓手是日志锚点。训练代码里在初始化、每个 step、每个 epoch 边界打印可识别的标记，NCCL 侧会自然产生 `NCCL INFO Bootstrap` 和 `NCCL INFO xxx Init COMPLETE` 这类行。这些锚点的价值在于对齐：把 256 个 Worker 的 `Init COMPLETE` 时间戳拉到一张表上，谁慢了 3 分钟一眼就能看出来，而这 3 分钟在指标图上是看不出来的——初始化阶段的 GPU 利用率本来就接近 0。

GPU 平面：四个利用率指标的读法

DCGM 里有四个容易被当成一回事、其实含义完全不同的利用率指标：GPU Utilization（DCGM_FI_PROF_GR_ENGINE_ACTIVE，一个周期内 Graphics/Compute 引擎处于 Active 的时间占比）、SM Active（DCGM_FI_PROF_SM_ACTIVE，至少一个 Warp 在 SM 上 Active 的时间占比，对所有 SM 取平均）、SM Occupancy（DCGM_FI_PROF_SM_OCCUPANCY，驻留 Warp 数与该 SM 最大可驻留 Warp 数的比）、Tensor Active（DCGM_FI_PROF_PIPE_TENSOR_ACTIVE，Tensor Core 处于 Active 的时间占比）。

这四个数的组合，能在完全不看业务代码的前提下给出性能判断。

表 2　四个利用率指标的组合读法。这张表的价值在于先定性再动手，避免拿网络当嫌疑犯查两周。

Utilization	SM Active	Tensor Active	判断	下一步
低	低	低	数据加载或 CPU 侧预处理占比过高，算力没喂饱	看 DataLoader、num_workers、/dev/shm、CPU 占用
高	低	低	通信占比高，GPU 在等数据	看 NVLink 与网卡吞吐、AllReduce 耗时
接近 100%	接近 0	接近 0	通信阻塞（NCCL Hang 或等待）	看 NCCL 日志、找掉队 Rank
高	高	低	模型中非矩阵计算占比过高	看算子构成，不是网络问题
高	高	超过 50%	算力已接近吃满，提升空间有限	停止优化，接受这个数

Util 打到 100% 而 SM Active 接近 0，是 NCCL 阻塞的特征指纹。这个组合看起来反直觉——GPU 明明 100% 忙碌，为什么是"卡住了"。原因是 Utilization 统计的是引擎有没有在被占用，而 NCCL 在等待对端时确实在轮询式地占着引擎；SM Active 看的是有没有真正在算东西，等待期几乎没有 Warp 在执行。判断方法很直接：把这张卡的指标和其他卡对比，如果全集群的卡都是这个形状，就是集体阻塞；如果只有一两张，那两张就是掉队节点。

采样间隔这里有一笔必须算的账。DCGM 常见的采集间隔是 10 秒，而一个训练 step 通常在几百毫秒到几秒之间，一次 AllReduce 在 8 卡 200Gbps 上只有百微秒到毫秒量级。也就是说，10 秒一个采样点对毫秒级事件是瞎的。结论有两条：第一，计数器类指标（Xid、PFC 帧、ECN 标记、CNP Slow Restart、CRC）必须按增量看，用 `increase()` 或 `rate()` 而不是瞬时值，瞬时值会把一次突发抹平；第二，事件型信息必须靠日志补，监控采不到的那部分只能靠锚点和 dmesg。还有一类典型盲区是 GPU

OOM——显存瞬间冲顶又释放，10 秒的采样点可能完全采不到，只能从用户日志里的 `OutofMemoryError: CUDA out of memory` 拿到。

表 3　GPU 平面的核心指标与采集命令。SM Clock 是最容易被漏掉的一个，而它恰恰是慢节点的主要信号。

指标	含义	采集命令	关注点	
Utilization / SM Active / Occupancy / Tensor Active	四个维度的忙闲程度	`dcgmi dmon -e 1001,1002,1003,1005`	组合判断，见上表	
SM Clock	卡的实际运行频率	`nvidia-smi -q -d CLOCK`	与基线比，掉到 1/2 即严重降频	
显存占用	显存水位与碎片	`nvidia-smi -q -d MEMORY`	OOM 风险与抢占判断	
Xid Error	硬件/驱动层错误码	`dmesg -T | grep -i xid`、DCGM_FI_DEV_XID_ERRORS	见第二节表 4	
NVLink 吞吐	机内卡间带宽	dcgm-exporter 的 NVLink 指标	判断 TP 是否真的在机内跑	
卡间拓扑	NVLink/PCIe 连接关系	`nvidia-smi topo -m`	交付后与变更前后各存一份	

网络平面：三个数必须常驻大盘

网络平面只需要三个数常驻大盘，其余按需下钻：PFC 暂停帧计数、ECN 标记率、headroom 余量。

PFC 暂停帧计数、ECN 标记率、headroom 余量这三个数必须常驻大盘，理由是它们分别回答无损网络的三个独立问题：PFC 计数回答"有没有发生反压"，ECN 标记率回答"拥塞控制有没有提前介入"，headroom 余量回答"反压发出去到生效之前，buffer 还撑不撑得住"。只看其中一个会得出完全错误的结论——PFC 计数在涨的同时仍有丢包，是 headroom 不足；ECN 标记一直在涨但吞吐不降，是 CNP 通道断了；PFC 计数长期为 0 而吞吐上不去，是流量压根没进无损队列。

第四个建议常驻的是 CNP Slow Restart（RoCE 场景一般对应 `roce_slow_restart_cnps`）。这个计数涨，说明发送端频繁进入慢启动重来，是链路质量差、降速或者网络侧有隐性丢包的强信号。它的价值在于能把"网络看着正常但任务慢"这种最难受的情况量化出来。

```
# ---- 主机侧：网络平面 ----
# PFC 帧收发计数，八个优先级分别统计，重点看无损优先级那一路
ethtool -S eth0 | grep -i -E 'pfc|pause'
# ECN 与 CNP 相关计数
ethtool -S eth0 | grep -i -E 'ecn|cnp'
# RDMA 端口硬件计数：重传、超时、乱序，正常应长期为 0
rdma stat show
# 看网卡当前协商速率，与预期（200G/400G）比对，防止降速
```

```
ethtool eth0 | grep -i speed

# ---- 交换机侧（以华为 VRP 体系为例，其他厂商命令名不同）----
# PFC 全局使能情况与无损优先级列表
display dcb pfc
# 单接口 PFC 配置与收发计数，接入侧与上行侧都要看
display dcb pfc interface 100GE1/0/1
# 队列统计：对比无损队列与缺省队列的报文计数增长
display qos queue statistics interface 100GE1/0/1
# ECN 标记与丢弃计数，压测时应持续增长
display qos queue statistics interface 100GE1/0/1 queue 3
# PFC 死锁检测记录
display dcb pfc deadlock
```

链路平面：物理层的四个证据

链路平面经常被跳过，但它决定了上面三层的数据可不可信。四个证据必须能拿到：**CRC 错包、模块收发光功率、BER/前向纠错情况、端口翻转次数。**

CRC 错包是最硬的那一个。一条健康链路的 CRC 错包计数应该长期为 0，出现即说明物理层已经出问题，而且是"无声"的——交换机不会把它算进拥塞丢弃，上层只会看到重传变多。光模块功率靠 `ethtool -m` 读 DDM 信息，接收光功率接近或低于模块标称的接收灵敏度，误码就是必然的，这类问题在夏天高温和光模块老化时集中爆发。端口翻转（flapping）要看事件而不是看瞬时状态，短到几秒的 down/up 在状态图上根本看不出来，但足以触发 NCCL 侧 `port error` / `port active` 的重连。

第四个证据是拓扑校验，交付后和每次网络变更后都要各存一份基线：`nvidia-smi topo -m` 校验卡间是不是完整的 NV8、`lldp` 校验每张网卡接到了预期编号的交换机端口。这两份基线是后面所有"拓扑变了"类问题的唯一对照物。

二、故障五类与定位路径

大规模集群的故障可以收敛成五类：硬件异常、通信故障、性能瓶颈、任务调度与资源利用、OOM。分类的意义在于先定性再动手——五类的定位成本差一个数量级，定错方向就是两周。

第一类：硬件异常

硬件异常的主要信号是 Xid Error。Xid 是 NVIDIA 驱动在运行中遇到硬件或驱动层错误时打出的错误码，带一个特定编号，可以通过 DCGM 的 DCGM_FI_DEV_XID_ERRORS 拿到一段时间内的最后一次错误码，也可以直接在 `dmesg` 里抓。

表 4　常见 Xid Error 速查。判断的第一刀是"要不要停卡报修"，这张表按这个分。

Xid	含义	节点侧日志特征	用户侧表现	处置
31	GPU Memory Page Fault	`MMU Fault`、`FAULT_PDE ACCESS_TYPE_VIRT_READ`	`an illegal memory access was encountered`	绝大多数是业务代码问题；极少数是硬件或驱动，可换卡复现
48 / 63 / 94	不可纠正 ECC（双位翻转）	`An uncorrectable double bit error`、`Row Remapper`	`uncorrectable ECC error encountered`	硬件问题，终止进程并重置 GPU，反复出现即报修
62 / 63	GPU 内部错误类	引擎/微控制器相关报错	常见 `unspecified launch failure`	复位 GPU；频繁出现需报修
74	NVLink 相关错误	NVLink 链路报错	通信异常、AllReduce 失败	检查 NVLink 与 NVSwitch，常需整机维护
79	GPU has fallen off the bus（掉卡）	`GPU has fallen off the bus`，常伴随 SXid 24007	`No CUDA GPUs are available`、`CUDA error: unknown error`	严重硬件故障，停卡报修
109 / 119 / 120	GSP RPC Timeout / GSP Error	`Timeout after 6s of waiting for RPC response from GPU x GSP!`、`errorString CTX SWITCH TIMEOUT`	`unspecified launch failure`	可考虑关闭 GSP；关闭后个别情况仍会单独出现 109

Xid 79 就是掉卡，没有第二种解释。看到它就走报修流程，不要试图复位重试。48/63/94 这类 ECC 错误的判断要更谨慎一点：一次单发的 ECC 可以复位后继续观察，但同一张卡一周内出现两次，就应该直接摘掉——万卡集群里一张反复报错的卡，代价不是这张卡的算力，而是它拖垮的每一次长任务重启。

Xid 之外还有三类硬件异常值得单独盯。

PCIe 降速和 GPU 降频都不会中断业务，只让任务变慢，这是它们最危险的地方。PCIe 降速在日志里的形态是 `PCIe 16x Link -> 2x Link`，速率可能从 252.048 Gb/s 掉到 31.506 Gb/s；用 `lspci` 看 PCI 总线配置，异常设备的 LnkSta 与正常设备明显不同，还会带 `(downgraded)` 标记。降速的卡不报错、不掉线，只是带宽变成八分之一，而跨机通信本来就只有机内 NVLink 的二十四分之一，再打个对折，这个节点立刻变成慢节点。GPU 降频同理，供电或散热问题会让 SM 时钟从 2GHz 掉到 500MHz，性能直接掉到四分之一。这类问题的唯一发现手段是基线比对：每台机器的 SM Clock、每张网卡的协商速率都要有基线值和自动比对。

第三类是频繁异常节点。相比慢节点，故障节点容易定位，但它对集群有效算力的杀伤更大——每一次故障都意味着一次重启和一段从上个 Checkpoint 到现在的算力浪费。Meta 在公开论文里把这类作业失败率显著高于平均水平的节点称为 Lemon 节点，通过识别并剔除它们，把 512 卡以上大规模作业的失败率从 14% 降到了 4%。集群生命周期里的故障率通常呈浴缸曲线：早期因为各系统没磨合好、频繁异常节点没被识别出来，故障很密集；中期稳定在恒定水平，故障随机；末期因为器件老化磨损，故障率重

新抬头。新集群上线头三个月应该按早期阶段管理，主动做节点筛选，而不是等故障来了才处理。

第二类：通信故障

通信故障里最高频的是 NCCL Hang 与 Timeout。默认超时是 30 分钟，日志里表现为 `Timeout(ms)=1800000`、`Some NCCL operations have failed or timed out`、`Connection closed by localRank`。缩短超时可以更快失败、更快重启，但太短会让一次网络抖动就直接把任务杀掉，这个取舍要按集群的实际抖动频率定。

定位路径按三个方向切。

第一个方向是通信操作不匹配。这是最常见的一类，代码里有逻辑分支导致一部分 Rank 走 AllReduce、另一部分走 AllGather，谁也等不到谁。判断依据在 Watchdog 日志里：`Watchdog caught collective operation timeout: WorkNCCL(SeqNum=xx, OpType::YYY, NumelIn=xxx, NumelOut=xxx, Timeout(ms)=60000)`。把全部 Worker 的 OpType 和 SeqNum 拉出来对齐，就能看出在哪一步分叉了。

第二个方向是物理层与链路。网卡抖动表现为短暂的 `Link down` / `Link up`，NCCL 侧对应 `port error` / `port active`，如果抖动够短，重试成功后任务会继续；如果网卡彻底不可用，就会走到 Timeout。

第三个方向是初始化阶段 Hang。这一类概率低但排查成本极高，常见原因与 `nvidia-fabricmanager` 有关——Fabric Manager 负责配置 NVSwitch 内存结构、协调 GPU 初始化、监控 NVLink 与 NVSwitch 错误。典型现象是 AllReduce 测试也 Hang，但设 `NCCL_ALGO=Ring` 就能跑，因为 NCCL 从 2.17 起支持的 NVLink Sharp（`NCCL_ALGO=NVSL`、`NCCL_NVLS_ENABLE` 默认开）依赖 Fabric Manager 正常。重启 fabricmanager 通常就能解决。

初始化阶段还有两类不属于 NCCL 但经常被归到"通信故障"里的问题：PyTorch 的 `MASTER_PORT` 被占用（`Address already in use`）和 `torch.distributed.DistStoreError: Socket Timeout`（Worker 与 Master 建连默认 300 秒超时）。后者最常见的原因是启动不同步——所有 Worker 在调用 torchrun 之前装依赖、编包，网络差异导致启动时间拉开五分钟以上。解决办法是在调用 torchrun 之前打印一条带 Rank 编号的日志，用时间戳就能判断谁没启动、谁晚了多少。

第三类：性能瓶颈

性能瓶颈里有三件事要分开：**确定性降速（PCIe 降速、GPU 降频）、慢节点、调度导致的降速。**

慢节点是大规模训练里最值得投入的一类。它的表现是同一个任务、同一套配置，多次启动的训练速度差 10% 以上，而这个差距往往超过很多优化方案的收益。字节在 MegaScale 的公开论文里也提到过慢节点，并且指出有些慢节点不是一开始就慢，而是在训练过程中逐渐变慢。定位慢节点靠两条路：一是用 Prometheus 加 Node Exporter 收集 CPU、GPU、PCIe 带宽指标做横向比对，缺点是差异可能很不显著；二是结合通

信时间分布（NCCL 里的 send/recv 耗时）看谁 consistently 落在尾部，后者更灵敏。

周期性降速是另一类隐蔽问题。表现是任务每隔一段时间掉一次速，怀疑过 DataLoader、怀疑过 Checkpoint、怀疑过节点上的定时任务，全部排除后，最终定位到某个 Rank 里 Python 的垃圾回收持有 GIL，导致这个 Rank 变成 Straggler；任务规模大时，多个 Rank 陆续变成 Straggler，影响被放大。Megatron-LM 里提供了主动 GC 的选项，在固定 step 后让所有 Rank 同时 GC，把各自的 GC 时间对齐到同一时刻，就能把这个抖动压下去。

调度导致的降速在第三节之前先提一笔：8 卡任务被调度到两台机器上，TP 需要跨机通信，速度直接差一倍。根因是细粒度调度没有做拓扑约束，这在第四节的第三板斧里展开。

第四类：任务调度与资源利用

集群通常不是专用于单一任务的。Meta 公开的数据是：集群里超过 90% 的任务规模小于一个节点（8 卡），但这批小任务只消耗不到 10% 的 GPU 时间，剩余算力被少数大任务占据。这个"少量大任务加大量小任务"的分布决定了调度策略必须双轨：大任务按节点粒度调度，整节点分配，保证通信效率；小任务按 GPU 粒度调度，允许多个小任务共享同一节点，避免碎片化。

按 GPU 粒度调度带来的副作用是抢占。如果隔离不彻底，用户强制使用非分配的 GPU 就会影响别人的任务——典型现象是监控里只有一个 GPU 的 SM Active 很高、其他都很低，定位后发现用户改了配置用了 0 号卡而实际分配的是 7 号卡。隔离靠 `CUDA_VISIBLE_DEVICES` 和 `NVIDIA_VISIBLE_DEVICES`，容器里还可以用 `NVIDIA_DRIVER_CAPABILITIES` 精细控制可用功能。

第五类：OOM

OOM 有四个变种，定位入口完全不同。Shared Memory OOM 的表现是 `Bus error: nonexistent physical address`，常见于 DataLoader 多进程共享内存不足，容器环境的 `/dev/shm` 默认偏小是一个高频触发点。GPU OOM 分三种：任务自身问题（PyTorch 会打印详细日志，包含已用显存和尝试分配的量，例如 `this process has 79.24GiB memory in use`、`Tried to allocate 3.38 GiB`）、抢占导致（某个 GPU 刚启动就占用明显偏高即可疑）、以及初始化型——PyTorch 用 lazy init 初始化 cuDNN，显存已经吃满时初始化会失败，抛出 `cuDNN error: CUDNN_STATUS_NOT_INITIALIZED`，这个报错里完全没有 "out of memory" 字样，最容易误判。NCCL 侧的 OOM 是 `NCCL WARN CUDA failure 2 'out of memory'`。Host OOM 比 GPU OOM 好查，`dmesg` 里通常有明确信息，常见于按 GPU 数量平均切分 Host Memory 之后，Master 进程全量加载或保存 Checkpoint。

显存碎片有一剂便宜的药：`PYTORCH_CUDA_ALLOC_CONF="expandable_segments:True"`，对输入 shape 频繁变化（图像尺寸、序列长度）导致频繁分配释放的场景效果明显，代价是可能影响分配性能。

表 5　五类故障的现象、定位命令与常见根因。按顺序从上往下过，能在十分钟内定性的就不要拖到第二天。

类别	典型现象	定位入口	常见根因			
硬件异常	任务中断、卡不可用、性能突降	`dmesg –T \	grep –i –E 'xid \	pcie\	link'`、`nvidia–smi –q`、`lspci`	Xid 错误、PCIe 降速、GPU 降频、网卡抖动、节点 Down
通信故障	NCCL Hang/Timeout、初始化卡住	NCCL 日志 OpType/SeqNum 对齐、`rdma stat show`、fabricmanager 状态	通信操作不匹配、网卡不可用、Fabric Manager 异常、启动不同步			
性能瓶颈	吞吐低、周期掉速、多次启动速度不一致	DCGM 指标横向比对、SM Clock 比对、通信时间分布	慢节点、PCIe 降速、降频、GC 导致 Straggler			
调度与资源	同样任务速度差一倍、单卡异常高负载	K8s Event、调度记录、显存占用分布	任务被拆到多机、GPU 隔离不彻底、碎片化			
OOM	进程被 Kill、CUDA out of memory、cuDNN 初始化失败	用户日志优先（监控采不到）、`dmesg` 查 Host OOM	Shared Memory 不足、显存碎片、抢占、Master 全量加载			

三、RoCE 网络的三个典型病

前面几篇把 PFC、ECN、headroom、DSCP 映射都配了一遍，这一节讲配完之后还会得的三种病。三种病的共同点是：交换机上的总丢包计数可能都是 0，但任务就是慢或者就是挂。

病一：PFC 风暴

表现是 PFC 暂停帧计数暴涨、上行口也被反压、连 SSH 和监控采集都跟着卡、吞吐掉一大截。定位时先看一件事：接入口的 PFC 计数涨了，上行口有没有跟着涨。跟着涨说明反压在逐跳扩散，这才是风暴；只在接入侧涨、上行侧干净，是正常的局部反压。

根因集中在三处。第一，开了过多的无损优先级——PFC 的暂停粒度是优先级，开得越多暂停面越大，一次 GPU 侧的 incast 就能把管理流量一起暂停。行业通识是只给 RoCE 数据优先级开 no–drop，其余全部可丢。第二，headroom 不足导致反压提前触发，反压又逐跳向上游扩散，最终形成大面积暂停域。第三，incast 规模本身过大：一次 AllReduce 的 ring 或 tree 阶段，几十上百个发送端会在毫秒级的时间窗里同时往一个接收端打数据，队列深度不是缓慢爬升而是瞬间冲高。

止血顺序是：先确认无损优先级是不是只有一个，再确认 headroom 是不是自适应、有没有覆盖当前链路距离，最后确认死锁检测有没有开。极端情况下可以临时把 PFC 关掉保住业务连通性，代价是丢包全部交给上层重传，这只能当临时手段。

CNP 那个优先级绝对不能开 no-drop。CNP 是 ECN 闭环里通知对端降速的信使，它一旦被暂停，闭环就断了。这个错误的隐蔽性在于你去看 ECN，标记计数在涨；去看 PFC，它也在工作；但吞吐就是上不去，重传计数一直在涨。

病二：ECN 失灵

ECN 失灵有两种相反的表现，判据也不同。

第一种是门限过高、来不及标记。等队列涨到门限才开始标记，CNP 走一个 RTT 回到发送端、降速生效，这段时间里队列还在涨，等降速起作用时可能已经丢包了。判据是：压测时吞吐上不去，而 ECN 标记计数几乎不涨，同时队列深度已经很高。

第二种是门限过低、误标记。门限定得太低，队列刚有点排队就开始标记，发送端无谓降速，吞吐长期上不去。在 200GE 端口上，如果排队 100KB 就开始触发标记，链路利用率可能长期卡在六成。判据是：ECN 标记率很高、吞吐上不去、但 PFC 计数和丢包计数都很干净。

门限到底该定在哪，取决于一条链路的带宽时延积。取一个具体算法：100GE 端口跨一跳，RTT 约 10 微秒，BDP 大约是 125KB；也就是说 headroom 至少要能吃下这个量级的突发，否则就是"反压发出去了但还是丢包"。200GE 时这个数字翻倍，跨机房长距链路还要再往上加余量。这就是为什么前面几篇里各家都推动态门限——静态门限无论拍在哪个值，都只是在"误标记"和"来不及"之间取折中，而不同模型、不同并行策略、不同批量下的最优门限还不一样。

第三种失灵是流量压根没进无损队列。判据最干脆：无损队列的报文计数为 0，而缺省队列在涨。根因来回复现过好几次——交换机接口漏配 `trust dscp`，或者主机侧没设 ToS（`NCCL_IB_TC=106` 对应 DSCP 26），报文按缺省优先级 0 进了尽力而为队列。

病三：无声丢包

第三种病最难受：交换机上看不到任何拥塞丢弃，PFC 和 ECN 计数都干净，端到端带宽测试也达标，但 `roce_slow_restart_cnps` 在涨、RDMA 重传计数在涨。这种丢包不发生在队列里，发生在物理链路上。

定位顺序是先易后难：交换机 `display interface` 看 CRC 错包与对齐错误，主机 `ethtool -S` 看 CRC 与各类错误计数，`ethtool -m` 读光模块 DDM 看接收光功率与温度，最后看端口翻转事件计数。一条健康链路的 CRC 应该长期为 0，出现即物理层问题；接收光功率接近或低于模块标称接收灵敏度，误码就是必然。

根因集中在光模块和光纤：模块老化或端面污染、光纤弯曲半径过小、跳线质量差、高温导致功率下降。这类问题在集群运行一年后会明显变多，和浴缸曲线的末期阶段是重合的。

表 6　RoCE 三个典型病的判据、定位与处置。三张表的共同判据是：先分清是队列问题还是物理问题。

病	判据	定位	处置
PFC 风暴	PFC 计数暴涨且逐跳扩散，非 RoCE 业务一起卡	对比接入与上行口 PFC 计数、看无损优先级数量	收紧到 1 个 no-drop 优先级、开死锁检测、核对 headroom
ECN 失灵	门限过高：标记不涨、吞吐上不去；门限过低：标记率高、利用率卡在六成	看 ECN 标记计数 + 队列深度 + 无损队列报文计数	按 BDP 重算门限或改用动态调优；核对 trust dscp 与主机 ToS
无声丢包	队列计数干净但 CNP Slow Restart 与重传在涨	CRC 错包、光模块 DDM、端口翻转计数	换光模块/光纤，核对收发光功率与温度

四、调优三板斧

调优的顺序不能乱：先集合通信，再数据加载与存储 IO，最后任务排布。理由是收益递减的反面——任务排布错了，前两项做到满分也补不回来。

第一板斧：集合通信调优

第一件事是把主机侧与交换机侧的 DSCP 对齐。交换机按 DSCP 26 映射本地优先级 3，主机就必须发出 DSCP 26 的报文；`NCCL_IB_TC=106` 就是 26 左移两位加 ECN 位。GID 索引要按 `show_gids` 的实际输出填，超时与重传次数给出一组常用起始值：

```
# ---- 主机侧 NCCL 常用环境变量（值需按版本与现网实测调整）----
# ToS：106 = 26<<2 + ECN 位，对应交换机侧 DSCP 26 → 本地优先级 3
export NCCL_IB_TC=106
# GID index 必须按本机 show_gids 的实际输出选 RoCEv2 那一行
export NCCL_IB_GID_INDEX=3
# RDMA 超时与重传：值越小失败越快，但网络抖动时也更容易误杀
export NCCL_IB_TIMEOUT=22
export NCCL_IB_RETRY_CNT=7
# 算法选择：NVLink Sharp 需 fabricmanager 正常，排查时可强制 Ring 做对照
export NCCL_ALGO=Ring
# 打开 NCCL Profiler 插件输出，用于提取集合通信性能数据
export NCCL_PROFILER_PLUGIN_ENABLE=1
```

第二件事是拓扑感知。NCCL 会读系统的 PCI 与 NUMA 信息构建拓扑，`nvidia-smi topo -m` 的输出就是它的输入之一：GPU 之间是 NV8 表示 8 条 NVLink，GPU 与网卡之间是 PXB（只跨 PCIe 交换芯片）、NODE（跨 PCIe 主机桥）、SYS（还要跨 NUMA）三档。这个输出必须在交付验收时存档，之后每次硬件变更重新采集比对——一张卡被换到另一个槽位，拓扑就从 PXB 变成 SYS，性能掉一截而没有任何报错。

第三件事是版本。有两个高频坑：NCCL 2.20.x 有 `Message truncated : received 1024 bytes instead of 256` 的已知问题，2.21 以上已修复；容器环境下 `Cuda failure 'invalid device context'`、`ncclUnhandledCudaError` 同样从 2.21 起修复。集群里 NCCL 版本不统一，是很多"同一套配置在不同机器上表现不同"的根源。

验收工具两件套：`ib_write_bw` 验点对点带宽与抖动，`all_reduce_perf -b 8M -e 4G -f 2 -g 1` 验集合通信。后者比前者更能反映真实训练，因为它把 incast 形态带进来了。

第二板斧：数据加载与存储 IO

这一板斧的全部内容可以概括成一句话：存储流量不能和训练流量抢同一张网。

典型 8 卡 A100 主机的内部拓扑是 2–2–4–6–8–8：2 片 CPU（NUMA）、2 张存储网卡、4 个 PCIe Gen4 交换芯片、6 个 NVSwitch、8 个 GPU、8 个 GPU 专属网卡。存储网卡直连 CPU，承担读训练数据、写 Checkpoint、SSH 管理和监控采集。GPU 专属网卡通过 PCIe 交换芯片连到各自的 GPU，专跑跨机集合通信。这是两套物理上分开的通路，对应两张网络。

混在一起的后果非常具体：Checkpoint 写入的突发流量打满 Leaf 上行、触发 PFC 反压，反压沿着无损优先级一路传到 GPU 训练流量上，整网降速，而根因看起来只是"有人存了个模型"。

Checkpoint 的账要算清楚，它决定了存储网要多大。以一个 70B 参数的模型做 bf16 训练为例：参数本身约 140GB，Adam 优化器状态按每个参数 12 字节算约 840GB，合计接近 1TB。如果不做分片，单个节点写 1TB 到存储，按存储网 2×100Gbps 折算 25GB/s 计算要 40 秒，这 40 秒里训练要么停要么被打满的上行反压拖慢。做了 ZeRO 类的分布式分片之后，假设 DP 并行度 128，每个节点只保存自己那一份分片，写出量降到 1TB/128 约 8GB，25GB/s 下约 0.4 秒。这就是分片保存和异步保存必须做的理由，不是"优化"，而是不做就跑不动。

Checkpoint 间隔是另一笔账：间隔越长，故障时的算力浪费越大（浪费量等于故障点与上个 Checkpoint 之间的全部计算）。大规模任务因为节点多、故障率高，很多团队把间隔压到 30 分钟甚至 5 分钟。压到 5 分钟的前提是把上面那笔分片账做完了，否则写 Checkpoint 本身就把训练拖死了。

最后一个细节是 Shared Memory。DataLoader 的多进程共享内存不足会直接报 `Bus error: nonexistent physical address`，容器里 `/dev/shm` 的默认值经常不够，需要在 Pod 规格里显式加大。

第三板斧：任务排布

第三板斧回答"为什么任务怎么摆影响这么大"，答案全在带宽账里。

表 7　链路带宽账与并行策略的落点。这张表是第三板斧的全部依据：通信量最大的那个并行维度，必须放在带宽最高的那层。

链路	单向带宽	相对机内	适合放什么
机内 GPU↔GPU（NVLink3，A100）	300GB/s（双向 600GB/s）	1 倍	TP，通信量最大
机内 GPU↔GPU（NVLink3，A800）	200GB/s（双向 400GB/s）	1 倍	TP

链路	单向带宽	相对机内	适合放什么
机内 GPU↔专属网卡（PCIe Gen 4）	32GB/s（双向 64GB/s）	慢约 9 倍	跨机通信的必经之路
跨机 100Gbps	12.5GB/s	慢 24 倍	DP 梯度同步、PP 激活传递
跨机 200Gbps	25GB/s	慢 12 倍	贴 PCIe Gen4 单向上限
跨机 400Gbps	50GB/s	慢 6 倍	须配 PCIe Gen5 才跑得满

先看这张表对应的数字。以 A100 平台为例：同主机 GPU 之间走 NVLink，A100 是 12 lane × 50GB/s = 600GB/s 双向、单向 300GB/s，A800 被阉到 8 lane，单向 200GB/s；同主机 GPU 到自己专属网卡走 PCIe Gen4 交换芯片，双向 64GB/s、单向 32GB/s；跨主机 GPU 之间看网卡，主流 100Gbps 折算 12.5GB/s。跨机比机内慢 24 倍。200Gbps 折算 25GB/s，已经贴近 PCIe Gen4 的单向带宽；400Gbps 折算 50GB/s，超过 PCIe Gen4 单向，必须配 PCIe Gen5 才发挥得出来。

这 24 倍直接决定了并行策略怎么落。TP（张量并行）的通信量最大，每次都要做 AllReduce，所以只要有条件就把 TP 限制在一台机器内走 NVLink，TP 的大小不要超过 8 这条经验就是这么来的。PP（流水线并行）只在阶段边界传激活值，通信量小得多，可以跨机，甚至跨 POD。DP 的梯度同步量介于两者之间，是 AllReduce 的主力。

调度上最常见的坑就是这条：一个 8 卡任务，调度到一整台机器上时，TP 全程走 NVLink，速度快；被拆到两台机器上时，TP 跨机，走 12.5GB/s 的网卡，速度差一倍。所以大任务必须按节点粒度调度，并且要有拓扑约束——不只是"8 张卡"，而是"同一台机器的 8 张卡"。

跨机部分还有一个摆放技巧，通常叫 rail-optimized：把一台机器上第 i 张卡的网卡接到同一台 Leaf，让一次跨机集合通信里同号卡之间的路径尽量短、跳数尽量一致。它的价值在于让 AllReduce 的 ring 上每一段的代价接近，否则快慢不一的段会让整环在最快的那段上也不能提速。

表 8　调优三板斧的收益、代价与验收方式。先做通信，再做存储，最后排布。

板斧	关键动作	预期收益	代价 / 边界	验收方式
集合通信	DSCP 对齐、GID 与超时参数、拓扑感知、NCCL 版本统一	消除大部分 Hang 与非必要重传	参数需按版本实测，调错会放大抖动	`ib_write_bw`、`all_reduce_perf` 带宽与抖动基线
数据加载与存储	存储网卡另走一张网、Checkpoint 分片与异步、加大 /dev/shm	消除周期性降速与写入反压	需要额外的存储网投资	训练期间计算网络 PFC 计数应为 0
任务排布	大任务节点粒度 + 拓扑约束、TP 限制机内、rail-optimized 放置	同样任务速度可差一倍	约束越强，调度碎片越多	同一任务多次启动的 MFU 方差

五、日常运维清单

最后落到可执行的东西。运维清单分日巡检、周巡检两档，外加容量水位和变更管理。

表 9　日巡检与周巡检清单。日巡检只做能在十分钟内出结论的动作，其余放周巡检。

频率	项目	看什么	异常处置
日	Xid Error	24 小时内新增 Xid 及编号	79/48 类立即摘卡；同卡两次以上 ECC 报修
日	SM Clock 比对	与基线偏离超过 10% 的机器	查供电与散热，摘出观察
日	网卡速率	协商速率是否等于预期	降速机器下线排查
日	PFC / ECN / headroom	三个常驻指标的趋势	见第三节三病表
日	CRC 与光功率	新增 CRC 错包、接收光功率趋势	物理层排查，换模块或光纤
日	Lemon 节点	各节点近 7 天任务失败次数排名	头部节点摘出单独诊断
周	拓扑基线比对	`nvidia-smi topo -m`、LLDP 邻居与存档一致	不一致即查硬件变更
周	带宽复测	`ib_write_bw`、`all_reduce_perf` 与基线比对	偏离超阈值即查网络变更
周	慢节点筛	全节点跑一遍基准压测，按耗时排序	尾部节点摘出诊断
周	版本核对	驱动、固件、OFED、NCCL 版本全集群一致	制定统一升级窗口

容量水位这块，建议把下面这组数字做成一张独立的看板，因为它决定了什么时候该扩容、什么时候该催用户清理。

表 10　容量水位与告警阈值建议。阈值是起始值，按自己集群的实际分布调。

水位	指标	建议阈值	说明
GPU 分配率	已分配 GPU / 总 GPU	超过 85% 关注	超过后大任务排队时间急剧上升
有效算力比	MFU 均值 / 目标 MFU	低于 80% 告警	比分配率更能反映真实利用情况
节点可用率	可调度节点 / 总节点	低于 95% 关注	摘卡过多会直接卡住大任务调度
网络重传率	RDMA 重传计数增量	长期应为 0	非 0 即链路或配置问题
CNP Slow Restart	`roce_slow_restart_cnps` 增量	持续增长告警	隐性丢包与降速的强信号
存储网水位	存储网利用率	超过 70% 关注	Checkpoint 突发需要余量

没有基线，前面所有现象你只能靠"感觉变慢了"。基线至少要留三组数，并且在交付验收阶段就采集存档：点对点带宽（`ib_write_bw`）、集合通信带宽（`all_reduce_perf`）、单节点训练的 MFU。这三组数的作用不只是"变慢了能发现"，更重要的是"变慢了多少、什么时候开始慢的"能说清楚，后者才是推动供应商和硬件团队处理问题的凭据。

变更管理只有一条硬规矩：任何网络侧和硬件侧变更，前后都要跑一遍基线测试并存档。绝大多数"运行三个月突然变慢"的问题，追到最后都是三个月里某次没人记录的变更。

最后说回这个系列。十篇里前八篇讲的是同一件事的不同解法——怎么把 RoCEv2 这条跨机通路做成无损：华为用 iLossless 把 ECN 门限和 headroom 交给算法动态调，华三用 DDC 把框式的控制面拆开，NVIDIA 用 Spectrum-X 把网卡和交换机一起设计，锐捷、中兴、Cisco、Arista、Juniper 各自在同一套 PFC 加 ECN 的骨架上加了不同的东西。第九篇把八家拉到一起横评，结论是骨架相同、差异在动态调优能力和可观测性上。第十篇也就是这篇，讲的是骨架搭完之后每天要面对的事。十篇串起来只说明一件事：无损网络不是配一次就结束的项目，它是一套需要长期维护的指标体系和处理流程。

这个系列到这里收官。感谢你跟着这十篇一路走完。从第一篇的 PFC 和 ECN 配置，到最后这篇的监控与故障定位，中间所有的命令和阈值都指向同一个目标：**让几万张卡在几个月里稳定地把算力交出去。**前面九篇的列表可以发送「系列」或「硬件」获取。 我是 **PingMaster**，深耕金融科技一线与 AI 基础架构。 本系列持续连载，发送「系列」或「硬件」获取完整文章列表，也欢迎私信交流。

#AI#GPU#RoCEv2#万卡集群#GPU运维#DCGM#Xid#NCCL#MFU#智算中心

喜欢作者

AI硬件架构 · 目录

上一篇 · RoCEv2 组网横评与选型决策：从千卡到万卡，一文抉择GPU集群组网：华为、华三...